

Technical Specification — Agentic Multimodal RAG

Status: Draft v0.9 · **Date:** 2026-09-29 · **Governed by:** 01-constitution.md · See the [changelog](#) for what changed in each version

1. Overview

The system ingests a corpus of PDFs, extracts every text passage, figure and table with provenance, and captions figures with a vision-language model (VLM) on a temporary GPU. It indexes everything for hybrid search. At query time, an agent (a LangGraph state graph calling OpenAI models) retrieves evidence with tools and returns a **structured multimodal answer**: text blocks, original images, generated charts and data tables, each with citations. Conversations are multi-turn, and a per-user long-term memory (semantic and episodic) personalises answers without ever serving as evidence.

1.1 Corpus (current)

File	Notes from initial inspection
Buildig-multimodal-rag.pdf	112 pages, ~30 embedded images, ~39 pages with heavy vector graphics (diagrams drawn as vectors, not images)
LLM_Training_and_Model_Lifecycle_Notes (2).pdf	57 pages, ~21 embedded images; first page has no extractable text (possibly scanned or image-based pages)
The_Complete_Guide_to_Post_Training_LLMs_v2_Expert_Edition.pdf	17 MB; not yet profiled
Transformers-in-Practice-Illustrated.pdf	0.8 MB; not yet profiled
The_Complete_Guide_to_Pre_Training_LLMs_v2_Expert_Edition.pdf (formerly pretraining.pdf)	2.3 MB; not yet profiled
retrieval_pipeline_preview.backup.svg	Not a PDF; out of scope for v1

A full corpus profile is the first task of Phase 1.

2. Requirements

2.1 Functional requirements

ID	Requirement
FR-1	Extract all text from each PDF with page number, bounding box and section heading path.
FR-2	Extract all embedded raster images at native resolution.
FR-3	Detect vector-drawn diagrams and render each diagram region to PNG at 200 DPI or higher.

ID	Requirement
FR-4	Extract tables as structured rows and columns, with headers and units.
FR-5	Detect pages with little or no extractable text and route them to VLM transcription.
FR-6	Generate a structured caption for every non-decorative figure using a VLM, informed by surrounding page context.
FR-7	Extract numeric data from charts found in PDFs, flagging each value as exact or estimated.
FR-8	Link figures and tables to the text that references them ("see Figure 3").
FR-9	Index text chunks, figure captions and tables for hybrid (dense + keyword) search.
FR-10	Provide an agent that plans, calls retrieval tools, checks evidence sufficiency and iterates.
FR-11	Return answers as ordered blocks: <code>text</code> , <code>image</code> , <code>chart</code> , <code>table</code> , <code>sources</code> .
FR-12	Generate bar, pie and line charts only from retrieved or computed data, always with the data table attached.
FR-13	Show original figures inline in answers, with source citations.
FR-14	Provide a chat UI that renders all block types, with clickable sources.
FR-15	Support incremental ingestion of new PDFs.
FR-16	Provide an evaluation harness with a golden question set.
FR-17	Require sign-in with email and password before any use of the application.
FR-18	Let an administrator create, disable, enable, unlock and reset user accounts (CLI).
FR-19	Enforce per-user daily question and cost limits.
FR-20	Keep per-user chat history, and a security audit log of sign-in events.
FR-21	Trace every question and every ingestion run end to end with OpenTelemetry: plan, agent rounds, tool calls, LLM calls, database queries, validation.
FR-22	Provide an admin view of usage, errors, latency and cost, with a link from any answer to its trace.
FR-23	Support multi-turn conversations: follow-up questions in a thread use the earlier turns, and a thread can be resumed later.

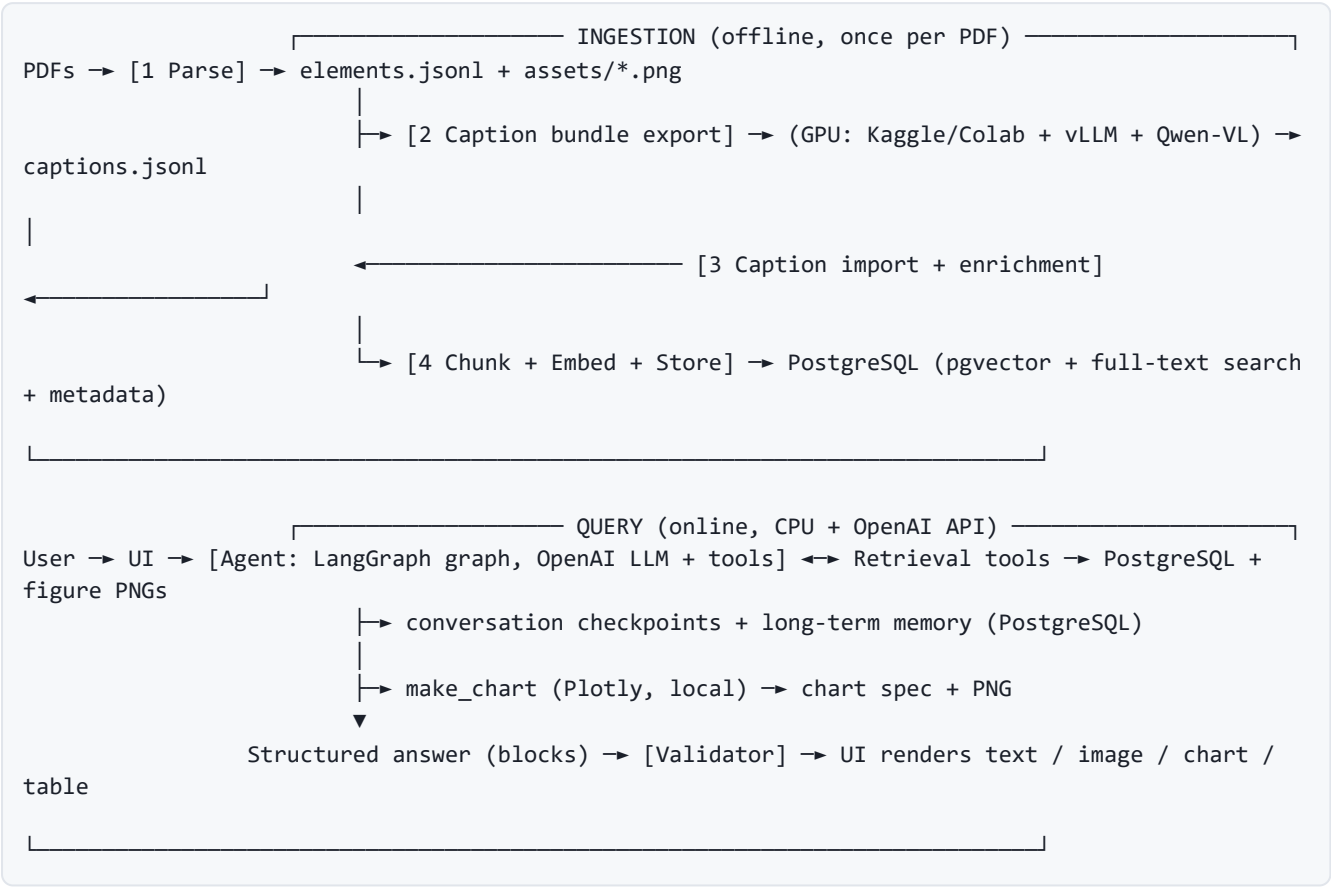
ID	Requirement
FR-24	Keep a per-user long-term memory: semantic (stable facts and preferences, e.g. "prefers charts over tables", "works on RLHF") and episodic (summaries of past conversations). Memory personalises and disambiguates questions; it is never cited as evidence (P13).
FR-25	Let each user see and delete their memories, and switch memory off.

2.2 Non-functional requirements

ID	Requirement	Target
NFR-1	Query latency (end to end)	Median ≤ 20 s; 90th percentile ≤ 45 s. Achievable only with parallel tool calls and the default 3-round limit (§7.2). Targets and round limits are confirmed together in Phase 6.
NFR-2	Query cost	Typical question $\leq$ US\$0.15
NFR-3	Query-time hardware	Laptop CPU only; no GPU
NFR-4	Ingestion GPU use	Temporary GPU session (Kaggle, Colab or rented)
NFR-5	Idempotency	Re-running ingestion on unchanged files makes zero VLM or embedding calls
NFR-6	Auditability	All intermediate outputs are human-readable (JSONL/PNG)
NFR-7	Portability	Runs on Windows 11; the GPU captioning job runs on Linux
NFR-8	Secrets	API keys and database credentials in <code>.env</code> only; never committed or logged
NFR-9	Consistency	A document's elements, captions, tables, chunks, vectors and keyword index are committed in one database transaction . Search never sees a half-ingested or mixed-version document.
NFR-10	Recoverability	Database backed up (e.g. <code>pg_dump</code> or managed snapshots) in production. The full index can be rebuilt from the PDFs plus cached captions and embeddings.
NFR-11	Credential security	Argon2id password hashes only; no plain-text passwords or session tokens stored or logged; generic sign-in errors; temporary lockout after repeated failures; HTTPS in production.
NFR-12	Telemetry safety	Telemetry never blocks or breaks the app; no secrets, passwords or tokens in telemetry; prompt and document content captured only when enabled (dev); retention limits applied (D13).
NFR-13	Memory privacy	Memories are scoped to one user, never shared, deletable by the user, excluded from telemetry content in production, and kept only for their retention period.

3. Architecture

3.1 High-level flow



3.2 Component responsibilities

#	Component	Responsibility	Runs on
1	Parser	Text blocks, images, vector diagrams, tables, textless-page detection	Laptop CPU
2	Caption bundle exporter	Packages figure PNGs and context into a portable bundle for the GPU job	Laptop CPU
3	VLM captioner	Runs Qwen-VL (vLLM, or transformers as fallback) and writes structured captions	Temporary Linux GPU
4	Enricher	Imports captions, validates them, links figures to text, normalizes tables	Laptop CPU
5	Indexer	Chunks text, embeds, writes each document to PostgreSQL in one transaction	Laptop CPU + embedding API
6	Retriever	Hybrid search in PostgreSQL (pgvector + full-text), RRF fusion, optional rerank	Laptop CPU + PostgreSQL
12	PostgreSQL	Single source of truth: metadata, captions, tables, vectors, keyword index	Local Docker (dev) / managed Postgres (prod)
13	Auth	Sign-in, sessions, lockout, per-user limits, audit events	App server + PostgreSQL

#	Component	Responsibility	Runs on
14	Observability	OpenTelemetry traces, JSON logs and metrics for every question and ingestion run	App + trace viewer (Phoenix/Jaeger)
7	Agent	LangGraph state graph (plan → tool rounds → compose → validate → repair) with OpenAI SDK calls inside the nodes; conversation checkpoints per thread	Laptop CPU + OpenAI API
15	Memory	Per-user semantic and episodic memory: extract after a turn, recall at planning; never evidence	Laptop CPU + PostgreSQL + OpenAI API
8	Chart engine	Validates data, chooses and renders the chart	Laptop CPU
9	Answer validator	Enforces constitution rules on the final answer	Laptop CPU
10	UI	Chat interface rendering the answer blocks	Laptop (Streamlit)
11	Eval harness	Runs the golden set and scores results	Laptop + OpenAI API

4. Technology stack

Concern	Choice	Rationale	Status
Language	Python 3.11 (already installed on the machine; 3.12 also fine), dedicated venv	System Python 3.14 may lack wheels for some libraries; 3.11 has wheels for everything used here (PyMuPDF, psycopg, pgvector, Streamlit, Plotly). The GPU job runs on Kaggle/Colab and is unaffected.	Decided
PDF parsing	PyMuPDF	Already installed; text with bounding boxes, images, vector drawings, <code>find_tables</code>	Proposed
VLM (primary)	Qwen2.5-VL-7B-Instruct, AWQ-quantized , on vLLM in fp16	Strong at diagrams and OCR. Full-precision 7B weights alone are ~16 GB and do not fit a 16 GB T4; the AWQ version does	Proposed
VLM (fallback A)	Qwen2.5-VL-7B-Instruct, 4-bit NF4 via Hugging Face <code>transformers</code> + <code>bitsandbytes</code>	Avoids vLLM install/CUDA issues; slower, small quality loss	Proposed
VLM (fallback B)	Qwen2.5-VL-3B-Instruct, fp16	Smallest footprint; lower quality on dense diagrams	Proposed
VLM serving	vLLM (offline batch) for the primary; plain <code>transformers</code> for fallback A/B	vLLM gives fast batching and structured (guided) JSON; fallback uses JSON mode + schema validation + repair	Decided
GPU host	Kaggle (free T4 16 GB, often 2×T4) or Google Colab; rented GPU as last resort	Needed only at ingestion. T4 limits: 16 GB VRAM each, no native bf16 (use fp16), so run the quantized 7B or the 3B model and cap image resolution (\$6.2)	Decided

Concern	Choice	Rationale	Status
Agent / answer LLM	OpenAI API, a vision-capable model with tool calling; model ID set in config	Owner has OpenAI access	Decided
Embeddings	OpenAI <code>text-embedding-3-large</code> at 1,536 dimensions (via the API's <code>dimensions</code> parameter) or local <code>bge-m3</code> (1,024) on CPU	Trade-off between cost/privacy and quality. Must stay $\leq 2,000$ dimensions , the limit of pgvector's HNSW index on the <code>vector</code> type (use <code>halfvec</code> if more are ever needed)	Open
Database	PostgreSQL 16+ with the pgvector extension: the single source of truth for metadata, captions, tables, vectors and keyword index	One transaction per document (NFR-9), SQL filters and joins, one system to back up and operate, no vendor lock-in	Decided
Semantic search	pgvector HNSW index, cosine distance; partial index per collection (or pgvector iterative scans) so filtered searches still return k results	Scales to millions of vectors, well beyond this corpus	Decided
Keyword search	PostgreSQL full-text search : two <code>tsvector</code> columns (<code>english</code> config for stemming, <code>simple</code> config for exact technical terms), GIN indexes, <code>ts_rank_cd</code> , <code>websearch_to_tsquery</code>	Built in, nothing to install. Upgrade path : ParadeDB <code>pg_search</code> (true BM25) if the Phase 6 evaluation shows weak keyword recall	Decided (upgrade: Open)
Fusion	Reciprocal Rank Fusion (RRF, $k = 60$), in one SQL query	Simple and robust; no weights to tune	Proposed
Reranker	Local cross-encoder (<code>bge-reranker-base</code>) on CPU	Optional, added in Phase 6	Proposed
DB hosting	Dev : official <code>pgvector/pgvector:pg16</code> Docker image (Postgres with pgvector preinstalled), with data in a named Docker volume , never a bind-mounted Windows folder (Postgres fails to start on NTFS permissions). Prod : managed Postgres with pgvector (e.g. AWS RDS, Azure, Supabase, Neon)	pgvector is not bundled with Windows Postgres installs; Docker is already on the machine	Proposed
DB access	<code>psycopg3</code> + <code>pgvector</code> Python package; schema changes as versioned SQL migration files	Plain SQL, easy to inspect (P9)	Proposed
Intermediate files	JSONL + PNG on disk (<code>data/</code>)	Audit trail of each ingestion stage (P8); Postgres is authoritative at query time	Proposed
Charts	Plotly (interactive in UI) + static PNG export (kaleido)	Hover shows exact values (P1)	Proposed
UI	Streamlit	Fastest route to a chat UI with images and charts	Open

Concern	Choice	Rationale	Status
Agent framework	LangGraph , for orchestration only: our own <code>StateGraph</code> ; nodes call the OpenAI SDK directly (no LangChain agents or chat-model wrappers, no prebuilt ReAct agent); tools stay plain Python functions; version pinned	Checkpointing (multi-turn threads, resume), a memory store, streaming of steps to the UI and human-in-the-loop interrupts come built in; the validator, round limits and citation rules stay in our code (P2, P4, P5); models stay a config change (P6)	Decided (D3)
Conversation state	LangGraph Postgres checkpointer in the project database	One store (P8a); threads survive restarts	Decided
Long-term memory	LangGraph Store on PostgreSQL + pgvector (semantic search over memories), per-user namespaces	One store (P8a); same embeddings as retrieval	Decided (D15)
Tool protocol	Tools are in-process Python functions. No MCP server (owner decision, D16)	Nothing outside the app needs the tools; avoids a process hop	Decided
Config	<code>.env</code> + a single config file	P6 swappability	Proposed
Authentication	Email + password accounts stored in PostgreSQL; Argon2id hashing (<code>argon2-cffi</code>); server-side sessions; admin-created accounts only	Owner's choice; no external identity provider needed; Streamlit's built-in login supports only OpenID Connect providers, so password sign-in is implemented in the app	Decided
Observability	OpenTelemetry (free, open-source standard): Python SDK with OTLP export; auto-instrumentation for the OpenAI SDK and pycpg; manual spans for the agent, tools, validator and ingestion	Vendor-neutral; each question becomes one inspectable trace of the agent's steps	Decided
Trace viewer	Dev: Arize Phoenix (free to self-host; trace UI built for LLM agents) in <code>docker-compose.yml</code> . Alternative: Jaeger (fully open source, generic). Prod: OpenTelemetry Collector → chosen backend	Swappable by config (P6)	Proposed (D12)
App logging	Python <code>logging</code> as JSON lines, each carrying <code>trace_id</code> / <code>span_id</code>	Logs link directly to traces	Proposed
Evaluation	RAGAS (open source, Apache-2.0), pinned version, with an OpenAI judge model; plus custom metrics for charts, figures, citations and refusals	Standard RAG metrics (faithfulness, relevancy, context precision/recall), multimodal and agent/tool-use metrics; custom checks cover what RAGAS doesn't	Decided
Testing	pytest , with markers <code>unit</code> , <code>integration</code> (a separate <code>mmrag_test</code> database in the Docker Postgres) and <code>live</code> (real OpenAI calls, run only on request); a mocked OpenAI client for unit tests; GitHub Actions CI (Phase 8)	Test-driven development where it fits (W5): fast, free and repeatable by default	Decided

Important: a ChatGPT subscription does not include API access. An OpenAI **API key with billing** is required.

5. Data model

5.1 Element (output of parsing): one record per extracted item

Field	Type	Description
element_id	string	Stable ID: {doc_id}:p{page}:{type}:{n}
doc_id	string	Short hash of the file content
source_file	string	Original filename
page	int	1-based page number
bbox	[x0,y0,x1,y1]	Position on the page (PDF points)
type	enum	text · image · vector_figure · table · scanned_page
section_path	list[string]	Heading hierarchy, e.g. ["3 Retrieval", "3.2 Hybrid search"]
text	string?	Text content (text elements)
asset_path	string?	PNG path (figures, scanned pages)
content_hash	string	SHA-256 of the content; used for caching and de-duplication
status	enum	ok · skipped · needs_review
skip_reason	string?	Required when status is skipped (P1)

5.2 Figure caption (VLM output): one per figure

Field	Type	Description
element_id	string	Links to the figure element
figure_type	enum	diagram · flowchart · chart · table_image · screenshot · photo · equation · decorative
short_caption	string	One line, shown under the image in the UI
detailed_description	string	Full description of components, labels and flow; this is what gets embedded
visible_text	list[string]	All text read from the image
extracted_data	object?	For charts: series, labels, values, units, and an exact / estimated flag per value
keywords	list[string]	Terms that help keyword search
confidence	enum	high · medium · low
model_id	string	VLM used (for reproducibility)

5.3 Table

Field	Type	Description
element_id	string	Links to the table element
columns	list[string]	Header names

Field	Type	Description
rows	list[list]	Cell values, kept as found
units	map	Column → unit, if known
numeric_columns	list[string]	Columns that can be charted
title / caption	string?	From the PDF when present
summary	string	Synthetic 2-sentence summary (§6.3), used for semantic search

5.4 Answer (agent output contract)

An answer is an ordered list of blocks:

Block type	Required fields
text	markdown, citations[]
image	element_id, asset_path, short_caption, citation
chart	chart_type (bar/pie/line), title, data (labels, series, values, units), value_provenance (exact/estimated per value), citations[]
table	columns, rows, citation
sources	list of {file, page, element_id}

Citations: agent vs final form.

- The **agent** outputs citations as {id} only, where id is either:
 - an **element_id**, for figures and tables (one element); or
 - a **chunk_id**, for text passages (a chunk can cover several elements).

The agent never outputs coordinates, file names or page numbers, which could be hallucinated.

- The **answer validator** (§7.3) looks up each id in PostgreSQL and hydrates the final citation. An **element_id** hydrates to one page + bbox:

```
{ "element_id": "...", "source_file": "Buildig-multimodal-rag.pdf", "page": 42, "bbox": [54.0, 120.5, 450.0, 380.0] }
```

- bbox is in PDF points (1/72 inch, origin top-left, as reported by PyMuPDF). This lets the UI draw a highlight immediately, without an extra lookup (see §7.5).
- A **chunk_id** hydrates to an **array of locations**, one page + bbox per element it covers, in reading order. A chunk can span several text blocks, or even two pages.

5.5 Database schema (PostgreSQL)

Table	Key columns	Purpose
documents	doc_id (PK), source_file, content_hash, version, status, ingested_at	One row per PDF version

Table	Key columns	Purpose
elements	element_id (PK), doc_id (FK, cascade delete), page, bbox (float[4]), type, section_path (text[]), text, asset_path, content_hash, status, skip_reason	Everything from §5.1
figure_captions	element_id (PK/FK), figure_type, short_caption, detailed_description, visible_text (text[]), extracted_data (jsonb), keywords (text[]), confidence, model_id, prompt_version	Everything from §5.2
doc_tables	element_id (PK/FK), columns (jsonb), rows (jsonb), units (jsonb), numeric_columns (text[]), title, summary	Everything from §5.3; served by get_table
figure_links	figure_id, text_element_id, method (explicit · deictic · proximity)	Figure-to-text links (§6.3)
search_chunks	chunk_id (PK), doc_id (FK, cascade delete), collection (text · figure · table), element_ids (text[]), dense_text, keyword_text, embedding (vector(N)), embedding_model, tsv_english and tsv_simple (generated tsvector columns)	What the search tools query (§6.4)
users	user_id (PK), email (unique, lower-case), password_hash, role (admin · user), status (active · disabled), must_change_password, failed_attempts, locked_until, created_at, last_login_at	Accounts (§7.6)
sessions	session_id (PK), user_id (FK, cascade), token_hash (unique), created_at, last_seen_at, expires_at, revoked_at	Server-side sessions
auth_events	event_id (PK), user_id (nullable FK), email_attempted, event_type, ip, created_at	Security audit log
query_log	query_id (PK), trace_id, user_id (FK), thread_id, question, rounds, tool calls, tokens, cost, latency, validator result, answer_json, created_at	Per-user history, cost and limits
LangGraph checkpoint tables	created by the checkpointer's own setup; keyed by thread_id	Conversation state per thread (FR-23)
LangGraph store tables	created by the store's own setup; namespace ("memories", user_id, kind) with pgvector index	Long-term semantic and episodic memory (FR-24)

Indexes on search_chunks :

- HNSW on embedding (cosine), partial per collection.
- GIN on tsv_english and on tsv_simple.
- B-tree on (collection, doc_id).

Deleting a documents row removes all of its data through cascading foreign keys.

6. Ingestion pipeline details

6.1 Parsing

1. **Profile** each PDF: page count, images, vector-heavy pages, tables, textless pages.
2. **Text:** extract blocks in reading order with bounding boxes. Detect headings (by font size and weight) to build `section_path`. Remove repeated headers and footers, logging each removal.
3. **Embedded images:** extract at native resolution. De-duplicate by hash. Filter out images below a size threshold (for example under 50×50 px) as `skipped: decorative`.
4. **Tables first:** detect tables with PyMuPDF's table finder **before** diagram detection, and store them as structured tables. Low-confidence tables are **also** rendered as images for VLM transcription.
5. **Vector figures:** find diagram regions from vector drawing operations, in this order.
 1. **Remove table regions.** Drop all drawing operations inside detected table bounding boxes, so cell borders and grid lines are never treated as diagrams.
 2. **Remove page furniture.** Drop header and footer rules and page borders: lines spanning more than about 85% of the page width, or very thin lines (aspect ratio above about 40:1) near the top or bottom margins.
 3. **Remove isolated simple shapes.** Drop a single rectangle or line with no other drawings nearby (underlines, dividers, bullets). A rectangle containing only body text is treated as a **callout box**, not a figure.
 4. **Keep box-and-arrow diagrams.** Do **not** require complex paths. Architecture diagrams (transformer blocks, RAG pipelines) are made of simple rectangles, arrows and text labels. A cluster qualifies when it has several nearby shapes and/or connecting lines, typically with text labels inside.
 5. **Cluster and render.** Cluster the remaining drawings, merge nearby clusters, include text labels inside the region, expand by a small margin, and render as PNG at 200 DPI or higher. Attach a nearby "Figure N" caption if present.

All thresholds (merge distance, minimum cluster size, width and aspect cut-offs) live in config and are **tuned against the Phase 1 review sheet**, not fixed up front.

6. **Textless pages:** any page with fewer than about 50 characters of text is rendered as a whole-page PNG and sent to the VLM for transcription and captioning.

6.2 VLM captioning (GPU, offline)

- **Bundle:** figure PNGs plus a `jobs.jsonl` file. Each job holds `element_id`, image path, page heading, the nearby paragraph (about 150 words before and after) and any "Figure N" reference text.
- **Prompt:** a type-aware instruction set. The model first classifies the figure type, then follows the matching instructions:
 - **Diagram:** list every component and arrow, and describe the flow.
 - **Chart:** give axes and series, and extract the values.
 - **Table image:** transcribe it into rows and columns.
- **Model options, chosen in the Phase 2 pilot:**
 - **Primary:** Qwen2.5-VL-7B AWQ on vLLM (fp16).

- **Fallback A:** Qwen2.5-VL-7B in 4-bit NF4 via `transformers` + `bitsandbytes`.
- **Fallback B:** Qwen2.5-VL-3B in fp16.

All three run the same prompt on the same 10 pilot figures. The best caption quality that runs reliably on the available GPU wins.

- **Image resolution cap:** set the processor's `max_pixels` (start at about $1280 \times 28 \times 28 \approx 1.0$ M pixels) so large figures don't exhaust GPU memory. The cap is raised only for dense diagrams whose labels become unreadable, and only if memory allows.
- **Output:** JSON matching §5.2.
 - **Primary path:** enforced with vLLM's structured-output (guided JSON) feature.
 - **Fallback path:** the prompt requests JSON; the output is validated against the schema (pydantic). Malformed output is repaired with a JSON-repair step or retried once, and otherwise marked `needs_review`.
- **Runtime:** a Kaggle or Colab notebook. It loads the chosen model, processes the bundle in resumable batches, writes `captions.jsonl`, and is downloaded back to the laptop.
- **Cache:** results are keyed by image `content_hash` plus model ID plus prompt version.

6.3 Enrichment and validation

- **Schema check:** validate each caption against the schema. Invalid or low-confidence captions are marked `needs_review`.
- **Label cross-check:** compare the VLM's `visible_text` against PDF text inside the figure's bounding box (for vector figures) and flag big mismatches.
- **Figure and table links:** link each figure **and each table** to the text that discusses it. The first rule that produces a link wins.
 1. **Explicit references:** "Figure 3", "Fig. 3", "Table 2".
 2. **Deictic phrases:** "as illustrated below", "the diagram above", "shown here". These link to the nearest figure or table in the indicated direction.
 3. **Most related nearby paragraph** (v0.9, at the owner's request): the candidates are the paragraphs directly before and after, and other paragraphs on the same page within about ± 150 points vertically. Each is scored by meaning (embedding similarity to the figure's caption or the table's summary) plus the share of the figure's own labels it contains. The best candidate above a configurable threshold is linked; if none passes, the item is reported as unlinked. A figure placed next to an unrelated paragraph is therefore not linked to it.
 4. Every figure also inherits the `section_path` of where it appears.
 5. Each link stores its rule and score.
- **Table summaries:** generate a short (2-sentence) synthetic summary of each table, for example: "Comparison of fine-tuning loss across LoRA, full fine-tuning and DPO on GSM8K and MMLU." Use a low-cost OpenAI model at enrichment time, cached by table hash.

6.4 Chunking and indexing

- **Text chunks:** section-aware, roughly 400–800 tokens, with about 15% overlap. A chunk never crosses a section boundary. Each chunk is prefixed with its `section_path`.
- **Figure documents:** `short_caption` + `detailed_description` + `visible_text` + `nearby paragraph`, pointing to the figure.

- **Table documents:** each kind of search gets different content, because they are good at different things.
 - **Semantic (dense_text → pgvector):** the synthetic summary + title + column names. Raw rows of numbers and acronyms embed poorly.
 - **Keyword (keyword_text → full-text search):** the title + column names + **all raw cell values**, so exact terms like "DPO" or "GSM8K" still match.
 - **Structured JSON:** stays in doc_tables and is served by get_table. It is never flattened for charting.
- **Text and figure documents** use the same content for dense_text and keyword_text.
- **Collections:** one search_chunks table with a collection column (text · figure · table) instead of separate indexes.
- **Keyword columns:** each row gets two generated tsvector columns from keyword_text.
 - english: stemmed, so "training" matches "trained".
 - simple: exact, so "GSM8K", "LoRA" and "text-embedding-3" survive intact.

6.5 Storage, consistency and versioning

- **Embed first, then write:** embeddings are computed (and cached) **before** the database transaction, so the transaction stays short.
- **One transaction per document:**
 1. Insert the new documents version.
 2. Insert all elements, captions, tables, links and search chunks.
 3. Delete the previous version (cascade).
 4. Commit.

On any failure, the whole transaction rolls back and the previous version keeps serving queries (NFR-9).

- **Idempotency:** re-ingesting an unchanged PDF (same content_hash) is a no-op.
- **Embedding model changes (blue-green):** every chunk records its embedding_model. To switch models:
 1. Backfill a new embedding column (or table) alongside the old one.
 2. Build its index.
 3. Switch the config.
 4. Drop the old column.

Search never mixes vectors from two models.

- **Schema migrations:** versioned SQL files in db/migrations/, applied in order.
- **Backups:** managed snapshots or nightly pg_dump in production (NFR-10).

7. Query-time design

7.1 Agent tools

Tool	Input	Returns
search_text	query, k	Top text chunks with citations

Tool	Input	Returns
<code>search_figures</code>	query, k	Top figures: short caption, description excerpt, element_id, citation
<code>search_tables</code>	query, k	Top tables: title, columns, element_id
<code>get_figure</code>	element_id	Full caption record, plus the image for LLM viewing if needed
<code>get_table</code>	element_id	Full structured table
<code>view_page</code>	file, page	Rendered page image, for when text extraction is insufficient
<code>compute</code>	expression over supplied numbers	Deterministic arithmetic result (safe evaluator, no free-form code)
<code>make_chart</code>	chart_type, title, labels, series, units, provenance, citations	Validated chart spec + PNG; rejects invalid input

Hybrid search (used by all three `search_*` tools): a single SQL query, restricted to the tool's `collection`.

1. **Semantic:** top 30 by cosine distance on `embedding` (HNSW).
2. **Keyword:** top 30 matching `websearch_to_tsquery` on `tsv_english` **or** `tsv_simple`, ranked by `ts_rank_cd`.
3. **RRF fusion:** each chunk scores $\sum 1 / (60 + \text{rank})$ across the two lists. Chunks found by only one list still count.
4. Return the top k (default 8). Each chunk's elements are **aggregated into one array** of `{element_id, page, bbox}` in reading order, so every chunk is **exactly one row**. A plain join would return one row per element, duplicating chunks and breaking the top- k count.
5. **Optional rerank** of the top ~ 20 with a local cross-encoder (Phase 6).
6. **Related items** (v0.9, at the owner's request): each result carries the elements linked to it (§6.3). A text result lists its linked figures and tables; a figure or table result lists its linked paragraphs. Related elements come along even when their own search missed them.

The query embedding is computed once per query and cached for identical queries.

7.2 Agent loop

Implemented as a LangGraph `StateGraph` (D3): nodes `recall_memory` $\rightarrow$ `plan` $\rightarrow$ `agent` $\rightleftharpoons$ `tools` $\rightarrow$ `compose` $\rightarrow$ `validate` $\rightarrow$ (`repair` $\rightarrow$ `validate`) $\rightarrow$ `remember`. Each conversation is a checkpointed thread (`thread_id`), so follow-ups see earlier turns (FR-23).

1. **Plan:** the agent classifies the question as conceptual, visual, quantitative, mixed or multi-part comparison, and decides which tools to use.
2. **Retrieve, with parallel tool calls:** independent tools are called **in the same turn**. For example, round 1 typically calls `search_text` + `search_figures` + `search_tables` together.
3. **Check sufficiency:** the agent asks whether the evidence answers every part of the question. If not, it reformulates and runs another round.
4. **Round limits:** a *round* is one LLM turn that emits tool calls.
 - **Default: 3 rounds.** Typically: parallel retrieve $\rightarrow$ inspect / compute / chart $\rightarrow$ final check.
 - **Up to 5 rounds** only for questions the planner classifies as multi-part or comparison.

- When the limit is reached, the agent composes with the evidence it has and states what is missing.
 - Final limits are set from the Phase 6 evaluation, together with NFR-1 and NFR-2.
5. **Compose:** the agent produces the answer in the block format of §5.4 (structured output), citing by `id` only (`element_id` or `chunk_id`, §5.4).
 6. **Validate:** the answer validator checks the result and hydrates citations (§7.3). On failure, the agent gets one repair attempt.
 7. **Memory:** before planning, the user's relevant memories are recalled; after a validated answer, new memories are extracted (§7.8).

7.3 Answer validator (constitution enforcement)

- Every block has at least one citation whose `id` exists (an `element_id` or a `chunk_id`). Unknown IDs fail validation.
- **Citation hydration:** for each valid `id`, the validator fills in `source_file`, `page` and `bbox` from PostgreSQL (`elements` table): one location for an `element_id`, and one per covered element for a `chunk_id`. Any coordinates or page numbers the LLM wrote itself are discarded.
- Every `image` block references an existing asset from the corpus (P3).
- Every number in a `chart` block appears in retrieved evidence or in a `compute` result (P4).
- Every chart carries its data table. Pie charts must represent parts of a whole.
- Charts with any estimated values are labelled "approximate".
- No citation points to a memory, and no fact in the answer is supported only by a memory (P13).

7.4 Chart engine rules

- **Pie:** at most about 8 slices, non-negative values, parts of a whole. Otherwise the engine falls back to a bar chart.
- **Bar:** categorical comparison with a single unit per axis.
- **Line:** ordered x-axis (time, steps, sizes).
- The chart shows exact values on hover and on labels, displays units, and has a source footnote.

7.5 Source highlighting (UI)

- The UI renders the cited page to an image at a known DPI (for example 150). It converts `bbox` from PDF points to pixels by multiplying each coordinate by `DPI / 72`.
- PyMuPDF's coordinate origin is top-left, the same as image pixels, so no axis flip is needed. A page with a `/Rotate` value must have the rotation applied first.
- The highlight is drawn as a semi-transparent rectangle over the rendered page.

7.6 Authentication and access (email + password)

- **Accounts:**
 - Created by an administrator through the CLI; there is no public sign-up.
 - Roles: `admin` and `user`.
 - A new or reset account gets a temporary password (shown once) and must change it at first sign-in.
- **Password storage:** Argon2id via `argon2-cffi`, with the library's recommended parameters. A hash is upgraded automatically at the next sign-in if the parameters change.

- **Password policy** (in line with NIST SP 800-63B):
 - at least 12 characters;
 - rejected if it appears in a common-password list;
 - no forced mixing of character types, and no periodic expiry.
- **Sign-in:**
 - Every failure shows the same message ("Invalid email or password, or too many attempts. Try again later."), so it never reveals whether an account exists.
 - Unknown emails are checked against a dummy hash, so every attempt takes the same time.
- **Brute-force protection:**
 - 5 failed attempts on one account within 15 minutes lock it for 15 minutes.
 - Failed attempts per client IP are also throttled.
 - Every attempt is written to `auth_events`.
- **Sessions:**
 - A random 256-bit token is created at sign-in; only its SHA-256 hash is stored in `sessions`.
 - Sessions expire after 8 hours idle or 7 days in total.
 - Logout, password change, password reset and account disable revoke all of the user's sessions.
 - The session is re-validated on every Streamlit rerun.
- **Session persistence (D10):** in v1 the token lives only in the browser tab's Streamlit session state, so a page refresh or new tab needs a new sign-in. This is simplest and safest: a cookie set from Streamlit can't be made HttpOnly (hidden from page scripts).
- **Per-user limits (D11):** before each question, today's question count and cost for the user are summed from `query_log`. Over the limit → a clear message, and the agent is not run.
- **Data visibility:**
 - Every signed-in user can query all documents in v1.
 - Each user sees only their own chat history.
 - Per-document permissions are out of scope for v1 (possible later with PostgreSQL row-level security).
- **Transport:**
 - Production must be served over HTTPS, with TLS at a reverse proxy in front of Streamlit.
 - Streamlit listens only on localhost.
 - Proxy headers are trusted for the client IP only behind that proxy.
- **Audit:** `auth_events` records sign-in success and failure, lockout, logout, password change and reset, and account creation and disabling, with time, user, email attempted and IP.

7.7 Observability (OpenTelemetry)

- **One trace per question.** The root span `query` contains child spans for:
 - the limit check and the plan;
 - each agent round, with every LLM call, tool call, embedding call and SQL query in it;
 - chart rendering, validation and any repair.

The trace's `trace_id` is stored in `query_log`, so any answer can be opened as a trace.

- **One trace per ingestion run** (`ingest.document`), with a span per stage: parse, bundle, enrich, chunk, embed, database write.

- **LLM spans** follow the OpenTelemetry GenAI conventions (`gen_ai.*`: model, input and output tokens), plus the cost in US\$.
 - **Logs:** JSON lines with `trace_id` and `span_id`, to standard output and a rotating file.
 - **Metrics:**
 - questions, errors, latency;
 - tokens and cost by model;
 - tool calls by tool;
 - validator failures;
 - failed sign-ins and lockouts;
 - ingestion time per stage.
 - **Content capture (D13):** full prompts, tool arguments and responses are recorded **in development only**. In production, only sizes, counts and IDs are recorded. Secrets, passwords, tokens and email addresses are never recorded; users appear only as `user_id`.
 - **Export:**
 - **Dev:** OTLP straight to Phoenix.
 - **Prod:** through an OpenTelemetry Collector, which batches data and samples it while keeping every error and slow trace.
 - **Retention (D13):** traces 14 days, app logs 30 days, `query_log` 90 days, `auth_events` 1 year.
 - **Failure safety:** export is batched and non-blocking. If the viewer is down, telemetry is dropped with a warning and answers are unaffected (NFR-12).
 - **Admin view (FR-22):** an admin-only page shows questions per day, error rate, slowest answers (linked to their traces), cost per user and failed sign-ins.
-

7.8 Long-term memory (semantic and episodic)

- **Semantic memory:** stable facts and preferences about the user, written as short statements ("prefers answers with charts", "is studying RLHF"). Updated rather than duplicated when a statement changes.
- **Episodic memory:** a short summary of each finished conversation: what was asked, what was found, what was left open.
- **Write path:** after a validated answer, a `remember` node asks a low-cost model to extract new semantic statements and, at the end of a thread, an episode summary. Stored in the user's namespace with an embedding.
- **Read path:** a `recall_memory` node searches the user's memories by meaning (top few) and passes them to the planner as **context about the user**, never as document evidence.
- **Rules (P13):** memories are never cited, never charted and never quoted as facts about the documents; they only shape what to search for and how to present it. A memory that contradicts the documents is ignored.
- **User control (FR-25):** each user can list and delete their memories and switch memory off; deleting an account deletes its memories.
- **Privacy (NFR-13):** per-user namespaces only; memory text is captured in traces only in dev; retention set in config.
- **Phasing:** conversation threads (checkpointers) arrive with the agent in Phase 4; long-term memory is built in Phase 9, after the UI.

8. Project structure (proposed)

```
Important-gen-ai-concept/
├── docs/                # constitution, spec, plan (this folder)
├── data/
│   ├── pdfs/           # source PDFs (current PDFs moved or linked here, with approval)
│   ├── elements/       # elements.jsonl per document
│   ├── assets/         # figure and page PNGs
│   ├── captions/       # captions.jsonl (from the GPU job)
│   └── cache/          # cached embeddings and table summaries (by content hash)
├── db/
│   └── migrations/     # versioned SQL schema files (tables, pgvector + GIN indexes)
├── docker-compose.yml  # local PostgreSQL + pgvector, and the Phoenix trace viewer (dev)
├── gpu_job/            # Kaggle/Colab notebook + bundle spec for VLM captioning
├── src/mmrage/
│   ├── ingest/         # parse, export bundle, import captions, enrich
│   ├── index/          # chunk, embed, transactional write to PostgreSQL
│   ├── db/             # connection, queries (hybrid search SQL)
│   ├── auth/           # passwords, sessions, sign-in, limits, audit events
│   ├── obs/            # OpenTelemetry setup: traces, JSON logs, metrics, retention cleanup
│   ├── retrieval/      # hybrid search, fusion, rerank
│   ├── agent/          # LangGraph graph, nodes, tools, prompts, answer schema, validator
│   ├── memory/         # semantic + episodic memory: extract, store, recall (Phase 9)
│   ├── charts/         # chart engine
│   └── config.py
├── app/                # Streamlit UI
├── eval/               # golden set, runner, reports
├── .env.example
└── README.md
```

9. Evaluation

Golden set: 40 questions, written with the owner:

Count	Question type	Example
15	Conceptual, text answer	"What is RLHF?"
10	Visual, figure expected	"Show the transformer block diagram"
8	Quantitative, chart expected	"Compare the dataset sizes mentioned for pretraining"
4	Mixed	—
3	Unanswerable	Checks P5 honesty

Each golden-set entry holds:

- the question and its type;
- a reference answer and reference evidence (element/chunk IDs);
- the expected figures;
- the expected chart values;
- the expected tool calls;
- whether it is answerable.

The set is a versioned file (`eval/golden_set.jsonl`). RAGAS's test-set generator may **draft** candidate questions from the corpus, but a question joins the golden set only after the owner reviews it.

Metrics. RAGAS scores are 0–1. Targets are provisional and confirmed after the baseline run.

Metric	Source	What it measures	Target (v1)
Faithfulness	RAGAS	Claims in the answer are supported by the retrieved evidence	≥ 0.90
Multimodal faithfulness	RAGAS	Same, for answers built on figures (image evidence)	≥ 0.85
Response relevancy	RAGAS	The answer addresses the question	≥ 0.85
Context precision	RAGAS	Retrieved evidence is relevant, ranked high	≥ 0.80
Context recall	RAGAS	The evidence needed for the reference answer was retrieved	≥ 0.80
Factual correctness	RAGAS	The answer agrees with the reference answer	≥ 0.75
Tool call accuracy	RAGAS (agent)	The agent called the expected tools	≥ 0.80
Chart numeric correctness	Custom	Every charted value matches its source	100%
Figure hit rate	Custom	The expected figure appears in the answer	≥ 80%
Citation accuracy	Custom	Cited elements belong to the reference evidence	≥ 90%
Correct refusal on unanswerable	Custom	Honest "not found in the documents"	3 / 3
Latency and cost	<code>query_log</code>	Per question	Within NFR-1 and NFR-2

How evaluation runs:

- `eval run` answers every golden question with the production configuration.
- RAGAS scores the answers with a fixed judge model (D14), and the custom checks run alongside.
- Results are stored in the database, and a report is produced with links to each question's trace.
- A drop beyond tolerance against the baseline, or any miss on a 100% metric, **fails the run**. No change is kept that fails it (P10).

Judge reliability:

- The judge model is fixed per baseline, the RAGAS version is pinned, and the baseline is run twice to measure judge variance.
 - The owner spot-checks 10 judgments per run.
-

10. Risks and mitigations

Risk	Impact	Mitigation
Vector-diagram detection splits or merges figures incorrectly, or mistakes table grids and rules for diagrams	Wrong or partial images	Remove tables and page furniture before clustering; keep box-and-arrow shapes; thresholds tuned on the review sheet; whole-page fallback (§6.1)
VLM does not fit or install on the free T4 GPU	Captioning blocked	AWQ 7B on vLLM → 4-bit transformers → 3B fp16; max_pixels cap (§6.2)
Agent latency exceeds NFR-1	Slow answers	Parallel tool calls; default 3-round limit (§7.2)
Tables missed by similarity search	Tables not retrieved	Synthetic summaries for semantic search; raw cells in the keyword index (§6.4)
pgvector unavailable in the Windows Postgres install	Setup blocked	Official pgvector/pgvector Docker image for dev; managed Postgres with pgvector for prod
Full-text search mangles technical terms ("GSM8K", "LoRA-r16")	Keyword misses	Second tsvector with the simple config; acronym queries in the Phase 3 retrieval tests
Postgres full-text ranking weaker than true BM25	Lower keyword precision	Measured in Phase 6; upgrade to ParadeDB pg_search if needed
Embedding dimensions above pgvector's 2,000 HNSW limit	Index can't be built	1,536-dim embeddings (or halfvec) (§4)
Filtered HNSW search returns fewer than <i>k</i> results	Missed evidence	Partial HNSW index per collection, or pgvector iterative scans
Partial writes during ingestion	Inconsistent search results	One transaction per document; rollback keeps the old version (§6.5)
Password guessing or credential stuffing	Account takeover	Lockout + per-IP throttling; common-password check; audit log (§7.6)
Stolen session token	Session hijack	Only token hashes stored; idle and absolute expiry; per-tab sessions (D10); HTTPS only
One user runs up API costs	Budget overrun	Per-user daily question and cost limits (§7.6)
Administrator forgets their password	No admin access	user reset-password from the CLI on the server
Prompts or document text leak into telemetry	Privacy	Content capture off in production; user_id instead of email; retention limits (§7.7)
Trace viewer down or slow	Lost traces	Batched, non-blocking export; answers unaffected (NFR-12)
GenAI telemetry conventions change (still evolving)	Attribute names drift	Names defined in one place (obs/ module)
LLM-judge scores are noisy or biased	Misleading quality numbers	Fixed judge model and pinned RAGAS version; variance measured on the baseline; owner spot-checks; hard custom metrics for charts and refusals
RAGAS API changes between versions	Broken evaluation runner	Pinned version; metric names checked against that version's docs in Phase 6

Risk	Impact	Mitigation
LLM emits invented coordinates or pages	Wrong highlights	Agent cites by <code>id</code> only (<code>element_id</code> or <code>chunk_id</code>); validator hydrates citations (§7.3)
VLM misreads chart values	Wrong charts	Values flagged <code>estimated</code> ; prefer exact values from text or tables; validator; "approximate" label
Python 3.14 incompatibilities	Install failures	Dedicated Python 3.11 venv (already installed)
Docker Postgres fails to start on Windows (data folder permissions)	Database unavailable	Named Docker volume for the data directory, never a bind-mounted Windows folder (§4)
Kaggle/Colab session limits	Interrupted captioning	Resumable batches; cache by hash
OpenAI model/API changes	Breakage	Model ID in config; thin client wrapper (P6)
Cost growth from agent loops	Budget overrun	Cap on tool rounds; token logging; cheaper model for sub-steps
Scanned pages with poor quality	Missing text	VLM transcription + <code>needs_review</code> flag
LangGraph API changes between releases	Broken agent after an upgrade	Version pinned; used only for orchestration (graph, checkpoint, store); agent-loop tests on a mocked OpenAI client
Memory presented as fact about the documents	Ungrounded answers	P13; memories passed as user context only; validator rejects memory citations; evaluation checks faithfulness
Wrong or stale memories	Misleading personalisation	Semantic memories updated in place; user can view and delete; memory can be switched off

11. Open decisions (owner to confirm)

#	Decision	Options	Recommendation
D1	Embedding model	OpenAI <code>text-embedding-3-large</code> (1,536 dims) vs local <code>bge-m3</code> (1,024 dims)	Decided: text-embedding-3-large at 1,536 dims (fixed in the database schema)
D2	UI	Streamlit vs Gradio vs CLI first	Streamlit
D3	Agent framework	Plain SDK vs LangGraph vs LlamaIndex	Decided (2026-09-29): LangGraph , orchestration only, OpenAI SDK inside nodes (§4). Changed from plain SDK at the owner's request, for multi-turn threads, long-term memory and step streaming.
D4	OpenAI model for the agent	Owner's available models	Decided: gpt-4o-mini (agent and table summaries; \$0.15 / \$0.60 per 1M tokens, about \$0.005 per question). Revisit if the Phase 6 evaluation shows quality gaps. Update 2026-09-29: table summaries moved to gpt-5.4-mini (owner's pick, Phase 3); the agent model is re-chosen by a side-by-side comparison in Phase 4.
D5	GPU host	Kaggle vs Colab vs rented	Kaggle (free, generous weekly quota)

#	Decision	Options	Recommendation
D6	Move PDFs into <code>data/pdfs/</code>	Move vs leave in place	Decided: moved to <code>data/pdfs/</code> (2026-09-26)
D7	Database	PostgreSQL + pgvector vs Pinecone + metadata DB	Decided: PostgreSQL + pgvector (single source of truth)
D8	Keyword ranking	Built-in full-text search vs ParadeDB <code>pg_search</code> (BM25)	Start built-in; decide after the Phase 6 evaluation
D9	Local Postgres setup	Docker <code>pgvector/pgvector</code> vs adding pgvector to the existing Postgres install	Docker (on a different port if the existing Postgres uses 5432)
D10	Session persistence	Per-tab session (sign in again after refresh) vs "remember me" cookie	Per-tab for v1 (a Streamlit-set cookie can't be HttpOnly)
D11	Per-user limits	Daily questions and cost	100 questions and US\$2.00 per user per day; adjustable in config
D12	Trace viewer	Phoenix (LLM-aware, free to self-host) vs Jaeger (fully open source) vs Grafana Tempo	Phoenix for development; confirm for production
D13	Telemetry content and retention	What telemetry may contain, and for how long	Content in dev only; traces 14 d, logs 30 d, query log 90 d, audit log 1 yr
D14	RAGAS judge model	Same model as the agent vs a different, capable OpenAI model	Decided: gpt-4.1 (different from the agent; fixed for each baseline)
D15	Long-term memory store	LangGraph Store on PostgreSQL vs a separate memory service	Decided: LangGraph Store on the project PostgreSQL (P8a)
D16	MCP server for the tools	Expose the tools over MCP vs in-process only	Decided (owner, 2026-09-29): no MCP server ; tools are in-process functions

12. Changelog

Version	Date	Changes
0.1	2026-09-26	Initial draft
0.2	2026-09-26	Review amendments. \$4/\$6.2: corrected the GPU error (full-precision 7B does not fit a 16 GB T4); added AWQ 7B (primary), 4-bit <code>transformers</code> (fallback A) and 3B (fallback B); <code>max_pixels</code> cap; JSON validate/repair on the fallback path. \$6.1: tables detected first; table regions, page furniture and isolated shapes removed before clustering; box-and-arrow diagrams kept; thresholds tuned on the review sheet. \$6.3: deictic and spatial-proximity figure linking; synthetic table summaries. \$6.4: table content split between dense, BM25 and structured storage. \$5.4/\$7.3: agent cites <code>element_id</code> only; validator hydrates <code>source_file</code> , <code>page</code> and <code>bbox</code> . \$7.2: parallel tool calls; 3 rounds by default, up to 5 for multi-part questions. \$7.5: new section on converting points to pixels for highlights. NFR-1 tied to the round limits. \$10: risks updated.

Version	Date	Changes
0.3	2026-09-26	Storage moved to PostgreSQL (replaces FAISS, rank-bm25 and SQLite). \$4: PostgreSQL 16+ with pgvector (HNSW) for semantic search; built-in full-text search (<code>english</code> + <code>simple</code> tsvectors, GIN) for keywords, with ParadeDB <code>pg_search</code> as the upgrade path; RRF in SQL; Docker for dev, managed Postgres for prod; embeddings capped at 2,000 dims (1,536 recommended). \$5.5: new database schema. \$5.4: chunk citations expand to several page/bbox pairs. \$6.4: single <code>search_chunks</code> table with <code>collection</code> column. \$6.5: new section on transactional ingestion, idempotency, blue-green embedding changes, migrations and backups. \$7.1: hybrid search query defined. NFR-9/10 added (consistency, recoverability). \$8, \$10, \$11 updated (D7–D9).
0.4	2026-09-26	Review fixes. \$4: Python 3.11 (installed); Docker data in a named volume. \$5.4/\$7.2/\$7.3: citations use one <code>id</code> (<code>element_id</code> or <code>chunk_id</code>); a chunk hydrates to an array of locations. \$7.1: hybrid search aggregates element locations, so each chunk is one row. \$10: Docker permission risk.
0.5	2026-09-26	Login added (owner chose email + password). FR-17 to FR-20, NFR-11; Auth component (§3.2); authentication row in \$4; <code>users</code> , <code>sessions</code> , <code>auth_events</code> and <code>query_log</code> in \$5.5; new \$7.6 (accounts, Argon2id, policy, logout, sessions, limits, visibility, HTTPS, audit); <code>auth/</code> package in \$8; four new risks; D10 and D11.
0.6	2026-09-26	Observability with OpenTelemetry. FR-21, FR-22, NFR-12; Observability component (§3.2); stack rows for OpenTelemetry, trace viewer and JSON logging (§4); <code>query_log.trace_id</code> (§5.5); new \$7.7 (trace per question and per ingestion, GenAI conventions, logs, metrics, content capture, export, retention, failure safety, admin view); <code>obs/</code> package and Phoenix in docker-compose (§8); three new risks; D12, D13.
0.7	2026-09-26	RAGAS evaluation. Stack row (§4); golden-set entry fields and RAGAS test-set drafting (§9); metrics table rewritten with RAGAS (faithfulness, multimodal faithfulness, relevancy, context precision/recall, factual correctness, tool call accuracy) plus custom metrics; regression gate; judge reliability; two new risks; D14.
0.8	2026-09-27	Testing row in \$4 (pytest, markers, test database, mocked OpenAI, CI), following constitution W5. Decisions D1, D4, D6 and D14 recorded as decided.
0.9	2026-09-29	\$6.3: tables linked too; link rule 3 becomes the most related nearby paragraph (meaning + shared labels) or unlinked. \$7.1: related items attached to search results. Agent on LangGraph (D3 changed): orchestration only, OpenAI SDK in nodes; Postgres checkpointer for multi-turn threads (FR-23). Long-term memory: FR-24, FR-25, NFR-13, new \$7.8, memory component, stack rows, schema rows, validator rule, risks, D15. No MCP server (D16). All at the owner's request.